Loop Engineering

JUNE 7, 2026

Loop engineering is replacing yourself as the person who prompts the agent. You design the system that does it instead. A loop here can be thought of a recursive goal where you define a purpose and the AI iterates until complete. I believe this may be the future of how we work with coding agents. However, its still early, I'm skeptical and you absolutely *have* to be careful about token costs (usage patterns can vary wildly if you are token rich or poor), so I want to unpack what it is and what it means.

Peter Steinberger recently said: "You shouldn't be prompting coding agents anymore. You should be designing loops that prompt your agents." Similarly, Boris Cherny, head of Claude Code at Anthropic, said "I don't prompt Claude anymore. I have loops running that prompt Claude and figuring out what to do. My job is to write loops".

Okay, so what does any of that mean?

For like two years the way you got something out of a coding agent was you wrote a good prompt and shared enough context. You type a thing, you read what came back, you type the next thing. The agent is a tool and you are holding it the entire time, one turn after the other. That part is kind of over, or at least some think it's going to be.

Now you build a small system that finds the work, hands it out, checks it, writes down what is done and then decides the next thing, and you let that system poke the agents instead of you. I wrote before about the cousin of this, agent harness engineering, which is making the environment one single agent runs inside and the factory model - the system that builds the software. Loop engineering sits one floor above the harness. The harness but it runs on a timer, it spawns little helpers, and it feeds itself.

The thing that surprised me is this is not really a tool thing anymore. A year ago if you wanted a loop you wrote a pile of bash and you maintained that pile forever and it was yours and only yours. Now the pieces just ship inside the products. Steinberger's list maps almost exactly onto the Codex app, and then almost the same onto Claude Code. And once you notice the shape is the same you stop arguing about which tool, you just design a loop that still works no matter which one you happen to be sitting in.

The five pieces, and then notes

A loop needs five things and then one place to remember stuff. Let me list it first and then map it.

1. **Automations** that go off on a schedule and do discovery and triage by themselves.
2. **Worktrees** so two agents working in paralell dont step on each other.
3. **Skills** to write down the project knowledge the agent would otherwise just guess.
4. **Plugins and connectors** to plug the agent into the tools you already use.
5. **Sub-agents** so one of them has the idea and a different one checks it.

Then the sixth thing, the memory. A markdown file, or a Linear board, anything that lives outside the single conversation and holds what's done and what is next. Sounds too dumb to matter. But it's the same trick every long running agent depends on and I went into it in long-running agents, the model forgets everything between runs so the memory has to be on disk and not in the context. The agent forgets, the repo doesnt.

Both products have all five now.

Primitive	Job in the loop	Codex app	Claude Code
Automations	discovery + triage on a schedule	Automations tab: pick project, prompt, cadence, environment; results land in a Triage inbox; `/goal` for run-until-done	Scheduled tasks and cron, `/loop`, `/goal`, hooks, GitHub Actions
Worktrees	isolate parallel features	Built-in worktree per thread	`git worktree`, `--worktree`, `isolation: worktree` on a subagent

Primitive	Job in the loop	Codex app	Claude Code
Skills	codify project knowledge	Agent Skills (`SKILL.md`), invoked with `$name` or implicitly	Agent Skills (`SKILL.md`)
Plugins / connectors	connect your tools	Connectors (MCP) plus plugins for distribution	MCP servers plus plugins
Sub-agents	ideate and verify	Subagents defined as TOML in `.codex/agents/`	Task subagents in `.claude/agents/`, agent teams
State	track what's done	Markdown or Linear via a connector	Markdown (`AGENTS.md`, progress files) or Linear via MCP

The names are a bit different here and there but the capability is the same thing. Let me go one by one because honestly the details are where a loop either holds together or quietly leaks everywhere.

Automations, this is the heartbeat

Automations are what make a loop an actual loop and not just one run you did once. In the Codex app you make one in the Automations tab and you pick the project, the prompt it will run, how often, and if it runs on your local checkout or on a background worktree. The runs that find something go to a Triage inbox, and the runs that find nothing just archive themselves wich is nice. OpenAI uses them internally for boring stuff like daily issue triage, summarising CI failures, writing commit briefings, hunting bugs somebody added last week. And an automation can call a skill, so you keep the recurring thing maintainable, you fire `$skill-name` instead of pasting a giant wall of instructions into a schedule that nobody will ever update.

Claude Code gets to the same place but through scheduling and hooks. You can run a prompt or a command on a interval with `/loop`, you can schedule a cron task, you can fire shell commands at certain points in the agent lifecycle with hooks, or you push the whole thing to GitHub Actions if you want it to keep running after you close the laptop. Same idea exactly, you define an autonomous task, you give it a cadence, and the findings come to you so you are not the one going around checking.

There is a second in-session primitive worth knowing, and it's the one closer to what this whole post is about. `/loop` re-runs on a cadence. `/goal` keeps going until a condition you wrote is actually true, and after every turn a separate small model checks whether you are done, so the agent that wrote the code isnt the one grading it. You give it something like "all tests in test/auth pass and lint is clean" and walk away. Codex has the same thing, also called `/goal`, it keeps working across turns until a verifiable stopping condition holds, with pause and resume and clear. Same primitive, both tools, wich is kind of the pattern for this whole article.

So this is the part that surfaces the work. The rest of the loop is what acts on it.

Worktrees so paralell doesnt turn into chaos

The second you run more than one agent the files start colliding, that becomes the failure. Two agents writing the same file is the exact same headache as two engineers committing to the same lines and nobody talked to each other first. A git worktree fixes it, its a separate working directory on its own branch sharing the same repo history, so one agent's edits literally can not touch the other one's checkout.

Codex builds the worktree support right in so several threads hit the same repo at once and dont bump into each other. Claude Code gives you the same isolation with `git worktree`, a `--worktree` flag to open a session in its own checkout, and a `isolation: worktree` setting you stick on a subagent so each helper gets a fresh checkout that cleans itself up after. I wrote about the human side of all this in the orchestration tax, the worktrees take away the mechanical collision but YOU are still the ceiling, your review bandwith decides how many you can actually run, not the tool.

Skills, so you stop explaining your project every single time

A skill is how you stop re-explaining the same project context every session like a goldfish. Both tools use the same format, a folder with a `SKILL.md` inside holding instructions and metadata, and then optional scripts, references, assets. Codex runs a skill when you call it with `$` or `/skills`, or by itself when your task matches the skill description, wich is the reason a tight boring description beats a clever one. Claude Code does it the same way and I wrote the pattern up in agent skills.

Skills are also where intent stops costing you over and over. I argued in the intent debt that an agent starts every session cold and it will fill any hole in your intent with a confident guess. A skill is that intent written down on the outside, the conventions, the build steps, the “we dont do it like this because of that one incident”, written one time where the agent reads it every run. Without skills the loop re-derives your whole project from zero every cycle, with skills it kind of compounds.

One thing to keep straight, the skill is the authoring format and a plugin is how you ship it. When you want to share a skill across repos or bundle a few together you package them as a plugin. True in Codex, true in Claude Code.

Plugins and connectors, the loop touches your real tools

A loop that can only see the filesystem is a tiny loop. Connectors, wich are built on MCP, let the agent read your issue tracker, query a database, hit a staging api, drop a message in Slack. Codex and Claude Code both speak MCP so the connector you wrote for one usually just works in the other. And plugins bundle connectors and skills together so your teammate installs your setup in one go instead of rebuilding the whole thing from memory.

This is the difference between an agent that says “here is the fix” and a loop that opens the PR, links the Linear ticket and pings the channel once CI is green by itself. The connectors are the reason the loop can act inside your actual environment instead of just telling you what it would do if it could.

Sub-agents, keep the maker away from the checker

The most useful structural thing in a loop, by far, is splitting the one who writes from the one who checks. The model that wrote the code is way too nice grading its own homework. A second agent with different instructions and sometimes a different model catches the stuff the first one talked itself into.

Codex only spawns subagents when you ask, runs them at the same time and then folds the results back into one answer. You define your own agents as TOML files in `.codex/agents/`, each with a name, a description, instructions and optional model and reasoning effort, so your security reviewer can be a strong model on high effort while your explorer is some fast read-only thing. Claude Code does the same with subagents in

`.claude/agents/` and agent teams that pass work between them. The usual split in both is one agent explores, one implements, one verifies against the spec.

I made this case twice already, once as the code agent orchestra and once as adversarial code review. The reason it matters specifically inside a loop is the loop runs while you are not watching, so a verifier you actually trust is the only reason you can walk away. Subagents do burn more tokens since each one does its own model and tool work, so spend them where a second opinion is worth paying for. This is also basically what Claude Code's `/goal` does under the hood, a fresh model decides if the loop is done instead of the one that did the work, the maker and checker split applied to the stop condition itself.

What one loop looks like

Stick it together and a single thread turns into a little control panel. Here is one shape I keep using.

An automation runs every morning on the repo. Its prompt calls a triage skill that reads yesterdays CI failures, the open issues, the recent commits, and writes the findings into a markdown file or a Linear board. For each finding that is worth doing the thread opens an isolated worktree and sends a sub-agent to draft the fix, and a second sub-agent reviews that draft against the project skills and the existing tests.

Connectors let the loop open the PR and update the ticket. Anything the loop can not handle lands in the triage inbox for me. The state file is the spine of the whole thing, it remembers what got tried, what passed, what is still open, so tomorrow morning the run picks up where today stopped.

And look at what you actually did there. You designed it one time. You did not prompt any of those steps. Thats Steinberger's whole point made real, and its the same loop in Codex or in Claude Code because the pieces are the same pieces.

What the loop still does not do for you

The loop changes the work, it does not delete you from it. And three problems actually get sharper as the loop gets better, not easier.

Verification is still on you. A loop running unattended is also a loop making mistakes unattended. The whole reason you split the verifier sub-agent from the maker is to make the loop's "its done" mean something, and even then "done" is a claim and not a proof. I keep saying the same line from code review in the age of AI, your job is to ship code you confirmed works.

Your understanding still rots if you allow it. The faster the loop ships code you did not write, the bigger the gap between what exists and what you actually get. Thats comprehension debt and a smooth loop just makes it grow faster unless you read what the loop made.

And the comfortable posture is the dangerous one. When the loop runs itself its very tempting to stop having an opinion and just take whatever it gives back. I called that cognitive surrender. Designing the loop is the cure when you do it with judgement and the accelerant when you do it to avoid thinking, same action, opposite result.

Build the loop. Stay the engineer.

I think this is a preview of how our work is going to evolve. That said, If I weren't reviewing the code myself or if I relied entirely on automated loops to fix it my product's quality would suffer. I'd likely end up stuck in a downward spiral, continuously digging myself into a deeper hole.

That said, go ahead and set up your loops, but don't forget that prompting your agents directly is also effective. It's all about finding the right balance.

Loops can also result in different outcomes depending on you. Two people can build the exact same loop and get completely opposite results. One uses it to move faster on work they understand deeply. The other uses it to avoid understanding the work at all. The loop doesn't know the difference. You do.

That's what makes loop design harder than prompt engineering, not easier. Cherny's point isn't that the work got easier. It's that the leverage point moved.

Build the loop. But build it like someone who intends to stay the engineer, not just the person who presses go.



Addy Osmani is a Software Engineer at Google working on Google Cloud and Gemini.

Tweet Bluesky Mastodon Threads LinkedIn Share

Want more? Subscribe to my free newsletter:

example@gmail.com Subscribe

Disclaimer: The views and opinions expressed on this site are my own and do not necessarily reflect the views, positions, or strategies of Google or any of its affiliates.

© Copyright 2026 Addy Osmani